

do same for spark without hadoop, install pyspark in virtual environments

Here is the complete step-by-step setup and execution guide for **PySpark without Hadoop**, installed inside a Python **virtual environment**.

TXT + 1

All system- or project-dependent placeholders that may change across systems or use cases are enclosed in `<< >>`.

TXT

Group A: Virtual Environment & Installation

Terminal 1: Environment Setup & Package Installation

1. Verify Prerequisites:

Bash



```
python3 --version[cite: 2]  
pip3 --version[cite: 2]  
java -version[cite: 2]
```

2. Create Python Virtual Environment:

Bash



```
python3 -m venv ~/<<spark-venv>>[cite: 2]
```

3. Activate Virtual Environment:

Bash



```
source ~/<<spark-venv>>/bin/activate[cite: 2]
```

4. Install PySpark inside Virtual Environment:

Bash



```
pip install pyspark==<3.5.7>[cite: 2]
```

5. Verify PySpark Installation:

Bash



```
pyspark --version[cite: 2]
```

Group B: Application Setup & Local Data Preparation

Terminal 1: Working Directory & Script Execution

1. Create Directory Structure:

Bash



```
mkdir -p ~/<spark-mapreduce-lab>/<input>[cite: 2]  
mkdir -p ~/<spark-mapreduce-lab>/<output>[cite: 2]  
cd ~/<spark-mapreduce-lab>[cite: 2]
```

2. Create Local Input CSV File:

Bash



```
nano <<input>>/<<employees.csv>>[cite: 2]
```

File Content:

Code snippet



```
<<emp_id>>,<<name>>,<<department>>,<<salary>>[cite: 2]  
101,Rahul,IT,60000[cite: 2]  
102,Priya,HR,45000[cite: 2]
```

```
103,Arun,IT,70000[cite: 2]
104,Neha,Finance,55000[cite: 2]
105,Kiran,HR,50000[cite: 2]
106,Anita,Finance,65000[cite: 2]
107,Ravi,IT,80000[cite: 2]
```

3. Verify CSV File:

Bash



```
cat <<input>>/<<employees.csv>>[cite: 2]
```

4. Create PySpark Script (Configured without Hadoop File System):

Bash



```
nano <<employee_salary.py>>[cite: 2]
```

File Content:

Python



```
from pyspark import SparkConf, SparkContext[cite: 2]

# Configure Spark to run locally using the local file system instead of HDFS
conf = SparkConf() \
    .setAppName("<<EmployeeSalaryAnalysis>>") \
    .setMaster("local[*]") \
    .set("spark.hadoop.fs.defaultFS", "file:///")[cite: 2]

sc = SparkContext(conf=conf)[cite: 2]

lines = sc.textFile("file:///home/<<ubuntu>>/<<spark-mapreduce-lab>>/<<input>>/<

header = lines.first()[cite: 2]
data = lines.filter(lambda line: line != header)[cite: 2]

mapped = data.map(
    lambda line: (
        line.split(",")[2],
        (int(line.split(",")[3]), 1)
    )
)
```

```

)[cite: 2]

# Grouping and aggregating department-wise salary sum and count
reduced = mapped.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
final_result = reduced.map(
    lambda x: f"{x[0]},{x[1][1]},{x[1][0]},{x[1][0]/x[1][1]}"
)

final_result.saveAsTextFile("file:///home/Ubuntu/spark-mapreduce-lab/output/department_salary.txt")

sc.stop()[cite: 2]

```

5. Clean Previous Output (If needed):

Bash



```
rm -rf output/department_salary.txt[cite: 2]
```

6. Submit Job via PySpark:

Bash



```
spark-submit employee_salary.py[cite: 2]
```

7. Verify Final Output:

Bash



```
cat output/department_salary/part-0[cite: 2]
```

Expected Output Format:

Plaintext



```

Finance,2,120000,60000.0[cite: 2]
HR,2,95000,47500.0[cite: 2]
IT,3,210000,70000.0[cite: 2]

```